PORTFOLIO · THEORY AND PRACTICE

QA: Questions and Answers

Denis Kurchikov, QA Engineer

59	9	1
questions with answers	topic sections	coding task with autotests

About this document

A collection of questions from technical tests I took when applying for manual QA, test automation and SDET positions. The questions are grouped by topic, each with an answer and a short explanation: why this option is right and how it differs from similar ones.

The document shows which topics I know and how I reason: from testing fundamentals and test design to networking, mobile platforms, automation and CI/CD.

Sections

- **1. Testing fundamentals** · 11
- **2. Test design** · 2
- **3. Bugs and test documentation** · 5
- **4. Web, network and browser** · 16
- **5. Mobile and cross-browser testing** · 4
- **6. API** · 1
- **7. Communication with developers and clients** · 3
- **8. Test automation and CI/CD** · 14
- **9. Python** · 3
- **Practice: coding task with autotests**

SECTION 1

Testing fundamentals

Testing types, principles, life cycle, severity and priority.

1. Regression testing is needed to check...

> **Answer:** that recent changes have not introduced new defects and that previously working functionality still works as before.

Checking a single fix is confirmation testing, checking a new feature is functional testing. Regression is about making sure changes did not break anything around them.

2. What is testing without documentation or pre-written test cases called?

> **Answer:** Exploratory testing.

The tester learns the product, designs scenarios and runs them at the same time, relying on experience.

3. A tester chewed a pencil, stuck it in a power outlet, soaked it and dried it. What type of testing is this?

Answer: Negative testing.

Every scenario uses the object in a way it is not meant to be used, to see how it behaves and where it breaks.

4. What is the earliest build version intended for internal testing called?

Answer: Alpha.

Alpha is tested inside the team, beta by a limited group of external users, and the release is for everyone. A debug build is not a release stage but a build type with debugging information.

5. What is a precondition in a test case?

Answer: The setup required to run the case: environment configuration, user registration, required data.

6. Which of these is not part of the software testing life cycle (STLC): planning, software improvement, results analysis, closure, bug fixing?

Answer: Software improvement and bug fixing.

These are development tasks. STLC stages: requirements analysis, planning, test case design, environment setup, test execution, closure. Results analysis is not a separate stage but is part of execution and closure.

7. Which definition best describes the pesticide paradox?

Answer: Tests become less effective over time.

Running the same tests over and over stops finding new bugs, so tests need regular review. Not to be confused with defect clustering: most bugs are concentrated in a small number of modules.

8. A new build has just been deployed to staging. What does QA do first?

Answer: A quick smoke test of the key scenarios.

First make sure the build works at all, then go deeper. Full regression as the first step takes too long.

9. A developer fixed a bug on the checkout page. What should be checked before release?

Answer: The entire checkout flow and related functionality.

Checking only the fixed button is not enough: the fix must not have broken neighboring steps. This combines confirmation and regression testing.

10. There is a typo in the company name in the site header, visible on every page. How do you classify it?

Answer: Low severity, high priority.

Functionality is not affected, so severity is low. But everyone sees it on every page, which is a reputational risk, so it must be fixed quickly.

11. An hour before release these bugs are found: a crash when opening a screen, data loss after saving a form, wrong tooltip text, 5-second page load against a 2-second target, wrong button color. Which are blockers?

Answer: The crash and the data loss.

A blocker makes the product unusable or irreversibly harms the user. Slow loading is serious, but the page works. Text and color are cosmetic.

SECTION 2

Test design

Techniques that cover more with fewer checks.

12. Which test design technique reduces the number of test combinations?

Answer: Pairwise testing.

Boundary values and equivalence partitioning reduce checks within a single parameter. Pairwise testing deals with combinations of several parameters: it covers all pairs of values without testing every combination.

13. A transfer of 10,000 RUB has no fee, while 10,001 RUB has a 1% fee. Which technique fits best?

Answer: Boundary value analysis.

There is a clear numeric boundary, so we test values around it: 9,999, 10,000, 10,001, 10,002. A decision table is for combining several conditions.

SECTION 3

Bugs and test documentation

Bug reports, choosing the right documentation, intermittent bugs.

14. Which bug title is best?

Answer: "Place Order button does not respond in Safari 17 / macOS at the checkout step".

The title immediately tells what is broken, where and in which environment. Emotional titles like "Nothing works!!!" are useless.

15. What must a bug report include?

Answer: Steps to reproduce, expected result, actual result, environment (browser, device, URL).

A guess about which line of code is broken does not belong in a report: QA describes observed behavior, and guesses can send the developer down the wrong path.

16. You need to quickly check a site after a minor WordPress plugin update. What fits best?

Answer: A short checklist of key pages and scenarios.

Full test cases and regression are overkill here, and checking only the plugin settings screen is too narrow: the plugin can affect the whole site.

17. A defect appears only after a specific sequence of actions and not every time. In what order do you act?

Answer: Retry several times with varied conditions → collect logs, screenshots, versions → ask a colleague to reproduce → check whether it was reported before → file the bug report.

Among the given options this is the most logical: the report is filed once the picture is clear. In practice I would check for duplicates first, so as not to spend time on a known bug.

18. In a mobile app, some users intermittently cannot send chat messages, and client and server logs show no errors. What is the most effective diagnosis?

Answer: Raise the server log level, test on different network types and with packet loss, look for a correlation with device models and OS versions, check request timeouts.

If the logs are silent, more data is needed. Chat depends heavily on the network, device fragmentation often explains bugs that affect only some users, and a timeout may not be logged as an error. Asking users is a weak first step: they cannot describe technical conditions.

SECTION 4

Web, network and browser

How web apps work, protocols, DevTools and typical diagnostics.

19. A button looks fine, but nothing happens on click. Which layer is most likely broken?

Answer: JavaScript.

HTML is the structure (the button exists), CSS is the look (it looks fine), and click behavior is JavaScript.

20. A form button does nothing. Console shows "Uncaught TypeError", and Network shows no request was sent. What is it?

Answer: A frontend (JavaScript) bug.

The request never reached the server, so the problem happened earlier, in the click handler, and the Console error confirms it.

21. You type "abc" into an email field and instantly, before submitting, see "Invalid email". Where does validation happen?

Answer: On the frontend, in JavaScript.

An instant reaction without a network request means the check runs in the browser. Server-side validation would require sending the data.

22. You logged in, closed the browser, and the next day you are still logged in. How does this work?

Answer: The session is stored in a cookie or local storage.

23. What are cookies and the browser cache?

Answer: Cookies are small pieces of data about the user and their visits that a site stores in the browser. The cache is temporary storage for already downloaded resources (images, scripts, styles).

24. A product image does not load. Where in DevTools do you look for the failed request and its status code?

Answer: In the Network tab.

Elements is the DOM, Application is storage, Sources is source code and JS debugging.

25. How do you emulate a slow connection in DevTools?

Answer: Network → the "No throttling" dropdown → choose a slow connection profile.

Network Link Conditioner is a separate macOS and iOS tool and is not part of browser DevTools.

26. A footer link shows "Page not found". Which status code do you expect in Network?

Answer: 404.

200 is success, 301 is a permanent redirect, 500 is an internal server error.

27. Which protocol is used for secure connections to websites?

Answer: HTTPS.

It is HTTP over TLS encryption. FTP is for file transfer, TCP is a lower-level transport protocol.

28. Which protocol is used for remote access to a server?

Answer: SSH.

HTTP is for the web, FTP for files, SMTP for email.

29. What happens if the DNS server is unavailable?

Answer: Sites will not open by domain name, but you can still connect by IP address.

DNS translates names into IP addresses. The internet itself keeps working, and the DNS cache lasts only a limited time.

30. What does the ping command do?

Answer: Checks whether a remote host responds and measures the response time.

31. What is an IP address?

Answer: A numeric identifier of a device on a network.

Not to be confused with a MAC address, the physical address of a network card.

32. What is a link that opens a specific section of an app or site called?

Answer: A deep link.

Unlike a regular link, it leads not to the home screen but deep into the app.

33. What is a suggest?

Answer: A dropdown list of suggestions while typing, for example search query options.

34. Which CSS selector selects paragraphs that come immediately after images inside an article?

Answer: article img + p

"+" selects the sibling immediately after. A space is a descendant at any level, ">" is a direct child, "~" is any following sibling, not necessarily adjacent.

SECTION 5

Mobile and cross-browser testing

Mobile platforms, browsers and traffic analysis.

35. What should you check in the mobile version of an online store?

Answer: Layout at different screen widths, tap target sizes, portrait and landscape orientation, behavior with the on-screen keyboard open.

Testing only on your own phone is the wrong approach: bugs are often specific to other screens and devices.

36. A slider works in Chrome but is broken in Safari. What is the right first step?

Answer: Reproduce in Safari, record the exact browser and OS version, check the console and file a bug with environment details.

37. How can you install an .ipa file on an iOS device?

Answer: Via Xcode, via an enterprise profile (Enterprise Distribution) and via Apple Configurator (iTunes was used for this in the past).

Simply sending an .ipa via AirDrop or opening it from Files does not work: iOS does not install such files without additional tools.

38. An app may be sending sensitive data over HTTPS to a third-party server. Charles Proxy is set up and the phone goes through it, but the app's traffic is not visible. Why?

Answer: The app uses certificate pinning.

The app trusts only a specific server certificate. The proxy substitutes the certificate, the app sees the mismatch and drops the connection, while the rest of the internet on the phone works.

SECTION 6

API

39. A complex nested GraphQL query returns 500 Internal Server Error, and there is no access to server logs. How do you find the cause?

Answer: In GraphQL Playground or Altair, gradually simplify the query by removing fields and nesting until the error disappears.

Without logs this is the only way to isolate the problematic field. Repeating the query only confirms reproducibility, and a bug report will still be needed, but with the cause identified.

SECTION 7

Communication with developers and clients

Gathering information and calmly resolving discrepancies.

40. A client writes: "The checkout button does not work". What do you do first?

Answer: Try to reproduce and ask for the browser, device, URL and a screenshot or video.

41. An issue reproduces for only one user. What do you collect?

Answer: Browser and version, device and OS, account ID and exact steps, a screenshot or screen recording, whether it reproduces after clearing the cache and in incognito mode.

Never ask for the user's password: it is not needed for diagnosis.

42. A developer says: "It works on my machine". What does QA do?

Answer: Reproduces again and shares the exact environment, steps and a short video with the developer.

Do not close the bug on their word, do not argue without evidence and do not escalate before understanding. Precise data shows the difference in conditions fastest.

SECTION 8

Test automation and CI/CD

Test pyramid, autotest stability, pipelines.

43. How do you prioritize test automation?

Answer: Automate first the scenarios that run often and at the same time have a strong business impact.

Frequent but non-critical scenarios waste resources, "easy to automate" is not a value criterion, and UI-only E2E tests are the most expensive to maintain.

44. The automation budget is small. Which level of the pyramid gets priority?

Answer: As many unit tests as possible, as few UI tests as possible.

Unit tests are the cheapest, fastest and most stable: maximum value for minimum cost.

45. How do you reduce autotest run time without losing quality?

Answer: Split tests by pyramid levels: run fast low-level tests often and fully, slow E2E tests selectively, keeping critical coverage.

46. The pipeline runs all 500 tests on every push and takes 40 minutes. How do you optimize it?

Answer: Run only smoke tests on feature branches and the full suite on merge to main.

Fast feedback on every push and a full check before code reaches the main branch. Splitting tests into two pipelines that run one after another gains nothing.

47. An hour before release a manager asks you to "quickly test" a fix. What do you do?

Answer: Run a fast regression set of autotests and clearly state which areas remain uncovered.

Make the most of automation and be honest about the risks, without a false sense of full coverage.

48. An autotest passes on DEV but consistently fails on STAGE, and the environments are "identical". First steps?

Answer: Rerun the test on STAGE several times, check server and browser logs on STAGE, compare test data and configuration between DEV and STAGE.

First rule out chance, then look for the real difference: "identical" environments rarely are. Filing a bug against development without diagnosis is unprofessional.

49. An autotest sometimes passes and sometimes fails with no code changes. Where do you start?

Answer: Add detailed logging (API responses, DB state, timestamps) and check whether the test depends on data left by other tests.

Shared state between tests is the most common cause of flaky tests. Disabling the test hides the problem instead of solving it.

50. After an unrelated feature is merged, CI fails: three different tests in random order with ElementNotInteractableException. First three actions?

Answer: Run the failing tests locally, check changes in the test repository over the last day, check whether the browser or web driver was updated on CI.

Random failures after an unrelated change point to infrastructure, such as an automatic driver update, rather than the feature itself.

51. What are the reliability problems in this test?

```
def test_login():
    driver.get("https://site.com/login")
    username_field = driver.find_element(By.ID, "username")
    password_field = driver.find_element(By.ID, "password")
    username_field.send_keys("admin")
    password_field.send_keys("password123")
    driver.find_element(By.ID, "login-btn").click()
    assert driver.current_url == "https://site.com/dashboard"
```

Answer: There are no explicit waits, and the login and password are hardcoded in the test.

The assert runs right after the click without waiting for navigation, a classic cause of flakiness. Test data should live outside the test code. ID locators, on the contrary, are among the most reliable.

52. An element appears a few seconds after the page loads. What is the most reliable way to interact with it?

Answer: Use an explicit wait for the element to appear.

Thread.sleep is a rigid pause, implicit waits mix poorly with explicit ones, and raising the global timeout slows down every test.

53. What are mocks?

Answer: Test doubles that let you control method calls and passed arguments and verify the expected behavior of the system under test.

Not to be confused with other doubles: a fake is a simplified working implementation, a spy records information about how it is used.

54. Which of these are mutations in mutation testing?

Answer: Replacing variables with others from the same scope, changing conditions in loops and branches, replacing arithmetic operators.

The code itself is mutated to see whether the tests catch it. Changing input data is test data work, not mutation.

55. How does Continuous Integration differ from Continuous Delivery?

Answer: CI is frequent merging of code into a shared branch with an automatic build and test run. CD extends CI: any build that passes the tests is ready to be deployed to production at any time.

56. How can QA contribute most effectively to feature development?

Answer: Get involved early: propose test scenarios, assess risks and influence the quality of requirements.

This is the shift-left approach: the cheapest bugs are the ones that never reach the code because they were caught at the requirements stage.

SECTION 9

Python

Reading code and working with lists.

57. What does this code print?

```
def find_even_numbers(numbers):
    return [num for num in numbers if num % 2 == 0]

print(find_even_numbers([1, 2, 3, 4, 5, 6]))
```

Answer: [2, 4, 6]

The list comprehension keeps only even elements.

58. What does this code with .append print?

```
def process_numbers(data):
    result = []
    for key, values in data.items():
        evens = [num for num in values if num % 2 == 0]
        if evens:
            result.append(evens)
    return result

data = {"a": [1, 2, 3], "b": [4, 5, 6], "c": [7, 8, 9]}
print(process_numbers(data))
```

Answer: [[2], [4, 6], [8]]

.append() adds the whole evens list as a single element, so the result is nested.

59. And if .append is replaced with .extend?

Answer: [2, 4, 6, 8]

.extend() adds elements one by one, so the list is flat.

PRACTICE

Coding task with autotests

Task: remove consecutive duplicates from a list and cover the solution with autotests. Any language allowed; the solution is in Python with pytest.

Approach: walk through the list and add a number to the result only if it differs from the last one added. Consecutive duplicates collapse, while non-adjacent repeats, e.g. [1, 2, 1], stay.

compress_numbers.py

```
def compress_numbers(numbers):
    """
    Removes consecutive duplicate numbers from a list.
    Example: [1, 1, 2, 2, 3] -> [1, 2, 3]
             [0, 0, 1, 1, 0] -> [0, 1, 0]
    """
    if not numbers:
        return []
    result = [numbers[0]]  # always keep the first element
    for num in numbers[1:]:
        # only add the number if it differs from the last one we kept
        if num != result[-1]:
            result.append(num)
    return result
```

test_compress_numbers.py

The tests cover the main scenario, the examples from the task, edge cases (empty list, single element, all elements equal) and the key requirement: non-adjacent repeats are not removed.

```
from compress_numbers import compress_numbers

def test_basic_duplicates():
    # Consecutive duplicates in the middle should collapse
    assert compress_numbers([1, 1, 2, 2, 3]) == [1, 2, 3]

def test_example_from_task():
    # Second example from the task description
    assert compress_numbers([0, 0, 1, 1, 0]) == [0, 1, 0]

def test_empty_list():
    # Edge case: empty input should return empty output, not crash
    assert compress_numbers([]) == []

def test_single_element():
    # Edge case: one element, nothing to compress
    assert compress_numbers([5]) == [5]

def test_no_duplicates():
    # No consecutive duplicates - the list stays unchanged
    assert compress_numbers([1, 2, 3, 4]) == [1, 2, 3, 4]

def test_all_same_element():
    # Every element is the same - collapses to a single value
    assert compress_numbers([7, 7, 7, 7]) == [7]

def test_non_consecutive_duplicates_are_kept():
    # Only consecutive duplicates are removed, not all duplicates
    assert compress_numbers([1, 2, 1, 2]) == [1, 2, 1, 2]
```